

# Property-based testing, partição e valor limite aplicados ao kata Hilbert's Hotel

Resenha crítica — Verificação e Validação, Trabalho T1

PUCRS / Escola Politécnica — 2026/I

Integrantes: Gabriel Bittencourt, Henrique Bueno e Mateus Neubarth

## 1 Introdução

O problema escolhido é o kata *Hilbert's Hotel* (Codewars, 7 kyu) (AZART\_\_, 2025). Um hotel com infinitos quartos está lotado e chegam novos hóspedes em ônibus. A função `hilbertsHotel(rooms, people, buses)` deve devolver o novo quarto de cada hóspede atual, de modo que todo recém-chegado ganhe um quarto distinto e nenhum quarto fique vazio. Há quatro regras: com pessoas e ônibus finitos, cada hóspede se desloca  $pe\tilde{c}o\tilde{c}o\tilde{c}o\tilde{c}o \times \hat{o}nibus$  quartos; com pessoas infinitas e ônibus finitos, o quarto é multiplicado por  $\hat{o}nibus + 1$ ; com ônibus infinitos e pessoas finitas, vale a mesma ideia; com ambos infinitos, usa-se o pareamento diagonal de Cantor, e o hóspede  $n$  vai para o quarto  $n(n+1)/2$ .

Implementamos o kata em JavaScript (Node 22) e testamos com Jest e fast-check. O kata é pequeno, mas o domínio mistura casos finitos e infinitos e uma fórmula não trivial, o que o torna bom para comparar testes por exemplos com testes por propriedades. O objetivo do estudo é responder: que técnicas dos artigos lidos são úteis neste SUT, o que cada uma detecta que a outra não detecta e como medir se os testes realmente servem?

## 2 Referencial

**Corgozinho, Valente e Rocha (2023)** analisaram como desenvolvedores escrevem testes baseados em propriedades (PBT) na prática. Partiram de 30 repositórios Python populares que usam Hypothesis, identificaram 763 testes PBT e classificaram manualmente uma amostra aleatória de 86 em dez categorias de propriedade. Dois padrões dominam: *roundtrip* (37,2%) e *test oracle* (33,7%), somando 70,9%. Quatro categorias não apareceram na amostra. Seis testes verificavam que uma chamada lança exceção, o que os autores sugerem ser uma categoria não descrita na literatura. Quanto ao uso da ferramenta, geradores (*strategies*) aparecem em 65,1% dos testes, *settings* em 25,5% e *assume()* em 15,5%. Pontos fortes: dados de código real e um catálogo de propriedades reutilizável como guia para iniciantes. Limitações: resultados preliminares, uma só linguagem e ferramenta, amostra pequena e classificação feita por uma única pessoa.

**Goldstein et al. (2024)** conduziram 30 entrevistas semiestruturadas com usuários experientes de PBT na Jane Street (OCaml). Os benefícios citados foram: confiança em código complexo, descoberta de casos de borda que o desenvolvedor não imaginou e propriedades como documentação executável que ajuda na revisão de código. Os entrevistados usam PBT de forma oportunista, em cenários de "alta alavancagem" onde a propriedade é óbvia: propriedades diferenciais (17 de 30), clássicas (11), *roundtrip* (11) e de falha catastrófica (7). As dores são escrever propriedades (16 de 30 disseram que isso atrasa), escrever geradores e *shrinkers*, e avaliar a eficácia dos testes: 11 de 30 admitiram não pensar o bastante se seus geradores exercitam o código, e só 7 usavam teste de mutação. Os testes PBT também precisam ser rápidos, com orçamentos entre 50 ms e 30 s. Pontos fortes: método qualitativo rigoroso, codificação temática validada e lista de oportunidades de pesquisa. Limitações: uma única empresa, linguagem e ferramenta, participantes experientes, saturação não medida e pesquisadores de PBT como entrevistadores.

**Aichernig e Schumi (2019)** propõem derivar automaticamente propriedades de PBT a partir de modelos de regras de negócio em XML de um serviço web industrial (TFMS, da AVL). Os modelos são traduzidos para máquinas de estados finitos estendidas (EFSM), e o FsCheck (C#) gera sequências aleatórias de comandos com dados de formulário. A propriedade verificada é que, após cada transição, o estado do modelo coincide com o do SUT. Em dois módulos, a abordagem revelou oito problemas confirmados pela AVL e atingiu cobertura de transições próxima de 100% com cerca de 4.000 casos no módulo maior, enquanto a cobertura de pares de transições chegou a apenas 75% com 5.000 casos. Pontos fortes: formalização, estudo industrial e uso de modelos que já existiam no sistema, sem modelagem manual. Limitações: o oráculo são as próprias regras, então só se acham desvios entre SUT e regras; as regras podem omitir restrições (problema

2 do estudo de caso); a geração aleatória não garante cobertura; e só um sistema foi avaliado.

**Comparação.** Os três tratam PBT como complemento, não substituto, de testes por exemplos. Corgozinho et al. mostram *o que* se testa (oráculo e *roundtrip*), Goldstein et al. explicam *por que* e *com que custos*, e Aichernig e Schumi mostram *como automatizar* a obtenção de oráculo e geradores a partir de uma especificação. Escolhemos esses três porque, juntos, orientam as propriedades, o processo de avaliação e a ideia de oráculo derivado da especificação.

### 3 Aplicação das técnicas no SUT

**Classes de equivalência e valor limite** (`tests/example.test.js`). O domínio foi particionado nas quatro regras do enunciado, nos casos degenerados (0 pessoas ou 0 ônibus), na lista vazia e nas entradas inválidas, resultando em 15 testes EP. Os 11 testes BV cobrem as fronteiras 0/1 de pessoas e ônibus, o intervalo de quartos que não começa em 1, o quarto 0, quartos não inteiros e o limite do inteiro seguro do JavaScript. Todo teste leva o ID da técnica no nome, e o `tests.md` traz a matriz de rastreabilidade entre técnica, regra do kata e teste.

**PBT** (`tests/property.test.js`, 11 propriedades, 300 execuções cada). Seguimos os padrões de Corgozinho et al. e as situações de alta alavancagem de Goldstein et al.: (i) *oráculo/diferencial*: `tests/oracle.js` enumera explicitamente cada passageiro e as diagonais de Cantor, sem usar as fórmulas fechadas, e verifica que o hóspede  $i$  cai no rótulo  $G_i$ ; (ii) *invariantes*: injetividade, saída estritamente crescente, função pura; (iii) *metamórficas*: trocar (infinitas pessoas,  $k$  ônibus) por ( $k$  pessoas, infinitos ônibus) não muda a saída, e duas chegadas finitas seguidas equivalem a uma com o total somado; (iv) *entradas inválidas*, a categoria "exceção esperada" que Corgozinho et al. observaram fora do catálogo. Da linha de Aichernig e Schumi aproveitamos a ideia de usar a especificação (aqui, os desenhos do enunciado) como modelo executável e oráculo, e a medição de cobertura para saber quando parar.

**Teste de mutação manual** (`scripts/mutation-check.sh`). Motivados pela observação de Goldstein et al. de que muitos desenvolvedores não interrogam propriedades que passam, injetamos 11 bugs (troca de operador, constante, condição e remoção de validação) e rodamos cada suíte isoladamente.

### 4 Discussão

**Resultados.** A Tabela 1 resume a execução.

| Indicador                                       | Resultado                                                                                             |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Testes                                          | 37 (15 EP + 11 BV + 11 PBT), todos passando                                                           |
| Cobertura (instruções, ramos, funções, linhas)  | 100%, com limite mínimo de 100% no Jest                                                               |
| Mutantes mortos (suíte completa)                | 11 de 11                                                                                              |
| Mutantes mortos só pelos testes de exemplo      | 2 (validação de quarto 0 e checagem de estouro): o gerador de propriedades nunca produz esses valores |
| Mutantes mortos por exemplos e por propriedades | 9                                                                                                     |

Tabela 1 — Resultados da suíte (fonte: os autores).

As propriedades detectaram todos os erros de fórmula, inclusive os que exemplos bem escolhidos também pegam, mas com muito menos esforço de escolha de entradas, o que confirma a promessa de Corgozinho et al. Já os testes de valor limite foram indispensáveis onde o gerador não chega, e essa limitação de distribuição de geradores é a mesma que Goldstein et al. relatam. Sem o teste de mutação, a suíte inteira passaria "verde" sem que percebêssemos esse buraco (OB6 do artigo). O mutante equivalente da primeira rodada mostrou que um ramo "ninguém chega" era código morto, e a remoção simplificou a solução.

**Dificuldades.** (1) O enunciado não define a saída exata nem o 3º caso para mais de uma pessoa por ônibus; interpretamos a saída como a lista dos novos quartos e generalizamos para  $\text{quarto} \times (\text{pessoas} + 1)$ . (2) Escrever bons oráculos independentes deu mais trabalho que a própria solução, o que ecoa o custo de especificar propriedades. (3) O kata não existe em JavaScript no Codewars, então não foi possível submeter a solução aos testes oficiais.

**Ameaças à validade.** Os oráculos e a solução foram escritos pelo mesmo grupo e compartilham a mesma leitura do enunciado, de modo que um erro de interpretação passaria despercebido. Os 11 mutantes foram escolhidos à mão e não representam todo o espaço de defeitos. As execuções de PBT usam sementes aleatórias, logo duas execuções exploram entradas diferentes. O SUT é pequeno e puramente funcional, um cenário de alta alavancagem que não generaliza para código com estado ou efeitos colaterais, onde Goldstein et al. relatam as maiores dificuldades. Por fim, os resultados dos artigos vêm de contextos distintos (Python, OCaml, C#) e os usamos como guia, não como evidência direta sobre JavaScript.

## 5 Conclusão

As técnicas se complementam. Classes de equivalência organizam o espaço de entrada e documentam as regras; valor limite cobre as fronteiras que geradores aleatórios raramente alcançam; PBT com oráculo explora combinações que ninguém escreveria à mão e verifica propriedades globais como injetividade e ausência de buracos. Lições aprendidas: (a) começar pelas propriedades de alta alavancagem (oráculo, invariantes, metamórficas) é o caminho de menor esforço; (b) medir a eficácia com cobertura e mutação é tão importante quanto escrever os testes; (c) quando a especificação é executável, pode-se derivar dela o oráculo, como defendem Aichernig e Schumi. Como trabalho futuro, pretende-se ampliar os geradores para incluir valores próximos aos limites e usar uma ferramenta de mutação automática.

## Referências

- AICHERNIG, Bernhard K.; SCHUMI, Richard. Property-based testing of web services by deriving properties from business-rule models. **Software & Systems Modeling**, v. 18, p. 889–911, 2019. DOI: 10.1007/s10270-017-0647-0.
- AZART\_. **Hilbert's Hotel**. Codewars, 2025. Disponível em: <https://www.codewars.com/kata/690743bd39f83ecf66068522>. Acesso em: 6 out. 2026.
- CORGOZINHO, Arthur Lisboa; VALENTE, Marco Tulio; ROCHA, Henrique. How developers implement property-based tests. In: IEEE INTERNATIONAL CONFERENCE ON SOFTWARE MAINTENANCE AND EVOLUTION (ICSME), 39., 2023, Bogotá. **Proceedings [...]**. [S. l.]: IEEE, 2023. Disponível em: <https://homepages.dcc.ufmg.br/~mtov/pub/2023-icsme-nier-pbt.pdf>. Acesso em: 6 out. 2026.
- GOLDSTEIN, Harrison; CUTLER, Joseph W.; DICKSTEIN, Daniel; PIERCE, Benjamin C.; HEAD, Andrew. Property-based testing in practice. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING (ICSE), 46., 2024, Lisboa. **Proceedings [...]**. [S. l.]: ACM/IEEE, 2024. Disponível em: <https://harrisongoldste.in/papers/icse24-pbt-in-practice.pdf>. Acesso em: 6 out. 2026.